Absolutely. Below is a **developer-oriented consolidated product-development summary** of Abraytech. It condenses the PRD into the functions, modules, users, pages, permissions, workflows, data responsibilities, and technical structure needed to actually build the MVP.

Abraytech — Consolidated Product Development Specification

1. Product Overview

Abraytech is a modular technology platform and corporate website for a technology business providing **software development, cybersecurity, artificial intelligence, IT consulting, technology education, digital services, technology products, and international education/travel support**. The platform should operate as a professional multi-page corporate website combined with a secure authenticated business application.

The recommended MVP architecture is a **Django modular monolith**. Django will provide the backend, authentication, business logic, database interaction, permissions, administration, APIs, and server-side rendering. **Tailwind CSS** will provide the design system and responsive UI, while **JavaScript** will handle interactive components and asynchronous functionality. PostgreSQL should be the production database, with Redis and Celery used for caching and background processing. Gunicorn and Nginx should handle production application serving.

The most important architectural principle is that Abraytech should be built as **one integrated platform rather than several disconnected websites**. A single user account should allow an individual to interact with different Abraytech services according to their role and permissions.

---

2. High-Level Application Structure

The recommended Django project structure is:

```
abraytech/
│
├── config/
│   ├── settings/
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
│
├── apps/
│   ├── core/
│   ├── accounts/
│   ├── permissions/
│   ├── website/
│   ├── store/
│   ├── orders/
│   ├── payments/
```

```
│   ├── lms/
│   ├── consultation/
│   ├── travel/
│   ├── repository/
│   ├── notifications/
│   ├── support/
│   ├── blog/
│   └── audit/
│
├── templates/
│   ├── base.html
│   ├── public/
│   ├── dashboard/
│   ├── accounts/
│   ├── store/
│   ├── lms/
│   ├── consultation/
│   ├── travel/
│   ├── repository/
│   ├── support/
│   └── admin/
│
├── static/
│   ├── css/
│   ├── js/
│   ├── images/
│   └── icons/
│
├── media/
└── manage.py
```

The developer should not create a completely separate frontend for every module. The entire application should use a common design system and a central `base.html`.

---

3. Central `base.html`

`base.html` is the foundation of the entire user interface.

It should provide the common:

- Header
- Navigation
- Logo
- Search
- Authentication controls
- Notification indicator
- User profile menu
- Main content area
- Flash messages
- Footer
- Global JavaScript
- Tailwind CSS

- Accessibility features

The navigation should be **role-aware**.

For example, a customer should not see administration links, while an instructor should see course-management functionality.

However, hiding a menu item is **not security**. Every protected Django view, API endpoint, action and object must perform server-side permission checks.

---

4. Main User Types

Abraytech should use a flexible RBAC system rather than hard-coding functionality directly against usernames.

User/Role	Primary Responsibility
Super Administrator	Complete system control
Administrator	General platform administration
Content Manager	Website, blog and project content
Store Manager	Products, inventory and orders
Finance Staff	Payments, invoices and financial records
Instructor	Courses, lessons, assessments and students
Admissions Officer	LMS applications and admissions
Student	Learning and course activities
Consultant	Consultation availability and appointments
Travel Advisor	Travel/study applications
Applicant	Submit and track applications
Customer	Store, consultation and general services
Support Agent	Customer support
Public Visitor	Browse public website

A single person may eventually have more than one role.

For example:

```
User
 ├── Customer
 ├── Student
 └── Consultant
```

Permissions should therefore be assigned independently from the user's identity.

---

5. Core Platform Module

The `core` application contains functionality shared by all other applications.

It should provide:

- Global settings
- Site configuration
- Common utilities
- Reference-number generation
- Shared status handling
- Common file utilities
- Common services
- Dashboard framework
- Error handling
- System health information
- Common constants

The core module should not contain business logic belonging specifically to Store, LMS, Travel or Consultation.

---

6. Accounts Module

The Accounts application controls identity and authentication.

Main functions

Function	Description
Registration	Create user account
Login	Authenticate user
Logout	End session
Email verification	Confirm email ownership
Password reset	Recover account
Profile	Manage personal details
Avatar	Upload profile image
Security	Manage password/session security
Account status	Active/inactive/suspended
User preferences	Notification and display preferences

The system should use a **custom Django User model from the beginning**, preferably with email as the primary login identifier.

---

7. Permission and RBAC Module

The permission system should use Django's permission framework as its foundation but organize permissions according to Abraytech's business modules.

Example:

```
store.view_product
store.add_product
store.change_product
store.delete_product

lms.view_application
lms.review_application
lms.approve_application
lms.reject_application
lms.issue_admission

consultation.view_booking
consultation.create_booking
consultation.cancel_booking

travel.view_application
travel.review_application
travel.request_document
travel.change_status
```

Permissions should be checked at three levels:

1. Navigation
2. View/API access
3. Individual object/action

The third level is particularly important.

For example, a consultant should only see their own consultation bookings, not every consultant's bookings.

---

8. Corporate Website Module

The website is the public-facing representation of Abraytech.

Its primary purpose is to:

- Establish credibility
- Explain services
- Generate leads
- Promote training
- Promote products
- Showcase projects
- Promote consultations
- Explain international education services

- Publish technology content
- Convert visitors into customers

---

9. Public Website Pages

The initial website should contain:

Page	Purpose
Home	Main company presentation
About	Company and leadership
Services	Technology services
Software Development	Software engineering services
Cybersecurity	Security services
AI & Data	AI/data services
IT Consulting	Consulting services
Training	Technology education
Store	Technology products
Consultation	Book professional services
Projects	Portfolio
Travel & Study	International education services
Blog	Technology articles
Careers	Employment opportunities
Contact	General enquiries
FAQ	Frequently asked questions
Privacy	Privacy information
Terms	Terms of service
Cookies	Cookie information

---

10. Homepage

The homepage should immediately communicate:

What Abraytech is, what it does, who it serves, and how a visitor can engage with it.

Recommended sections:

1. Hero
2. Company introduction
3. Core services
4. Technology capabilities
5. Featured projects
6. Cybersecurity
7. AI/data

8. Training
9. Store
10. Consultation
11. Travel/study
12. Testimonials
13. Latest articles
14. Call to action
15. Footer

Primary calls to action should include:

- Start a Project
- Book Consultation
- Explore Training
- Visit Store
- Explore Projects

---

11. Store Module

The Store provides e-commerce functionality for Abraytech technology products and gadgets.

The basic workflow is:

```
Product
   ↓
Product Details
   ↓
Cart
   ↓
Checkout
   ↓
Payment
   ↓
Order
   ↓
Processing
   ↓
Shipping
   ↓
Delivery
```

Store functions

Function	Customer	Store Manager
Browse products	✓	✓
Search products	✓	✓
Filter products	✓	✓
View product	✓	✓

Function	Customer	Store Manager
Add to cart	✓	—
Checkout	✓	—
Pay	✓	—
View orders	✓	✓
Create products	—	✓
Edit products	—	✓
Manage inventory	—	✓
Manage categories	—	✓
Process orders	—	✓

---

12. Store Webpages

Public/customer pages

```
/store/
/store/category/<slug>/
/store/product/<slug>/
/cart/
/checkout/
/orders/
/orders/<reference>/
```

Management pages

```
/dashboard/store/
/dashboard/store/products/
/dashboard/store/products/create/
/dashboard/store/products/<id>/edit/
/dashboard/store/categories/
/dashboard/store/orders/
/dashboard/store/inventory/
```

---

13. Store Data

Core models:

```
Category
Product
ProductImage
ProductSpecification
Cart
CartItem
Order
OrderItem
```

Products should have SKU, price, stock, category, images, descriptions and status.

Historical orders must retain product information such as name, SKU and unit price so that changing a product later does not corrupt historical records.

---

14. Orders Module

Orders should be separated from Products because an order represents a financial transaction, whereas a product represents inventory.

Recommended statuses:

```
PENDING_PAYMENT
PAID
PROCESSING
SHIPPED
DELIVERED
COMPLETED
CANCELLED
REFUNDED
```

Customers should be able to see their own orders.

Store managers should be able to manage orders according to their permissions.

Finance staff should be able to see payment information without necessarily having permission to modify inventory.

---

15. Payments Module

Payments should be centralized because Abraytech will eventually accept payments for several services.

Payments may relate to:

- Store orders
- Courses
- Consultations
- Travel services
- Other future services

The payment system should integrate with Stripe.

The critical principle is:

A browser redirect must never be treated as proof of payment.

Stripe webhooks should verify payment status on the server.

---

16. LMS Module

The LMS is one of the most strategically important applications.

It should function as an online technology training platform where users can discover courses, apply, receive admission, enroll and learn.

The lifecycle is:

```
Course Discovery
      ↓
Application
      ↓
Application Review
      ↓
Decision
      ↓
Admission
      ↓
Enrollment
      ↓
Learning
      ↓
Assessment
      ↓
Completion
      ↓
Certificate
```

---

17. LMS User Types

Role	Main Functions
Public Visitor	Browse courses
Applicant	Apply and track application
Admissions Officer	Review applications
Instructor	Teach/manage courses
Student	Learn and complete assessments
Administrator	Full LMS management

---

18. LMS Course Management

Instructors/admins should be able to create:

```
Course
```

```
├── Module
│    ├── Lesson
│    ├── Lesson
│    └── Assessment
├── Module
└── Module
```

Courses should support:

- Title
- Description
- Category
- Level
- Duration
- Price
- Requirements
- Learning outcomes
- Instructor
- Modules
- Lessons
- Assessments
- Publication status

---

19. LMS Application System

Applicants should be able to:

1. Create account.
2. Select course.
3. Complete application.
4. Upload required documents.
5. Save application as draft.
6. Submit application.
7. Track application.
8. Respond to requests for additional information.
9. Receive decision.
10. Receive admission letter where approved.

---

20. LMS Application Status

```
DRAFT
SUBMITTED
UNDER_REVIEW
INTERVIEW_REQUIRED
ACTION_REQUIRED
APPROVED
```

```
REJECTED
WITHDRAWN
```

Every important status change should be recorded in a history table.

---

21. Admission System

After approval, authorized admissions staff should be able to issue an admission.

The system should generate:

- Admission reference
- Admission letter
- Course/programme
- Student details
- Admission date
- Start date
- Conditions
- Issuing officer

The admission document should be generated by the backend and securely stored.

---

22. Student Portal

Once admitted/enrolled, a student should have access to:

```
Student Dashboard
├── My Courses
├── Lessons
├── Assignments
├── Quizzes
├── Progress
├── Results
├── Certificates
├── Payments
├── Messages
├── Notifications
└── Profile
```

A student must not be able to access another student's private information.

---

23. LMS Assessment

The MVP can support:

- Multiple-choice quizzes
- Assignments
- Instructor grading
- Feedback
- Scores
- Completion tracking

Future versions can introduce:

- Practical assessments
- Coding challenges
- Projects
- AI-assisted tutoring
- Live examinations

---

24. Certificate System

After meeting course completion criteria, the system should generate a certificate.

The certificate should contain:

- Student name
- Course
- Certificate number
- Date
- Abraytech branding
- Verification code
- QR code

A public verification page should allow an employer or third party to verify the certificate without exposing sensitive student information.

---

25. Consultation Module

The Consultation application allows users to book professional technology services.

Potential services include:

- Cybersecurity consultation
- Software architecture consultation
- AI strategy
- Digital transformation

- Technology career guidance
- IT consulting

---

26. Consultation Workflow

```
Select Service
      ↓
Select Consultant
      ↓
View Availability
      ↓
Select Slot
      ↓
Enter Details
      ↓
Payment
      ↓
Booking Confirmation
      ↓
Meeting
      ↓
Completion
```

The backend must prevent double booking.

Availability should consider:

- Consultant working hours
- Existing bookings
- Holidays
- Blocked dates
- Special working hours
- Time zones

---

27. Consultation Pages

```
/consultation/
/consultation/services/
/consultation/service/<slug>/
/consultation/consultants/
/consultation/consultant/<id>/
/consultation/book/
/consultation/confirmation/
/dashboard/bookings/
/dashboard/bookings/<reference>/
```

---

28. Consultant Portal

A consultant should be able to:

- Manage profile
- Define availability
- Block dates
- View bookings
- View client information permitted for the session
- Add consultation notes
- Mark booking completed
- Reschedule where authorized

A consultant must not have unrestricted administrative access.

---

29. Travel & Study Module

The Travel module should initially focus on **international education and study application support**, rather than trying to implement every possible travel service from the beginning.

Its objective is to provide a structured digital case-management system.

---

30. Travel Applicant Functions

Applicants should be able to:

- Create profile
- Add academic information
- Select destination
- Select programme
- Create application
- Upload documents
- Submit application
- Track status
- Receive document requests
- Receive notifications
- Communicate with assigned advisor

---

31. Travel Advisor Functions

Travel advisors should be able to:

- View assigned applicants

- Review profiles
- Review documents
- Request missing documents
- Add internal notes
- Update application status
- Record institution information
- Record application progress
- Communicate with applicants

Sensitive internal notes should not be visible to applicants.

---

32. Travel Application Status

```
DRAFT
SUBMITTED
UNDER_REVIEW
DOCUMENTS_REQUIRED
PROCESSING
SUBMITTED_TO_INSTITUTION
OFFER_RECEIVED
COMPLETED
REJECTED
WITHDRAWN
```

The system should make the workflow configurable enough to support future services.

---

33. Travel Document Management

Documents should be stored securely.

Potential documents include:

- Passport
- Academic certificate
- Transcript
- CV
- Personal statement
- English-language evidence
- Financial documents
- Institution correspondence

Documents must have access restrictions.

A normal customer should never be able to access another applicant's documents by changing an ID in the URL.

---

34. Repository/Portfolio Module

The Repository showcases Abraytech's technical work.

This is important not only for marketing but also for demonstrating technical capability to prospective clients, partners and institutions.

Each project should contain:

- Project title
- Summary
- Problem
- Solution
- Features
- Industry
- Technology stack
- Images
- Results
- Case study
- Publication status

---

35. Repository Pages

```
/projects/
/projects/<slug>/
/projects/category/<slug>/
```

Admin/content manager:

```
/dashboard/projects/
/dashboard/projects/create/
/dashboard/projects/<id>/edit/
```

---

36. Blog/Content Module

The blog should support technology-focused content.

Potential categories:

- Cybersecurity
- AI
- Software Development
- Django

- Cloud
- Programming
- Technology Careers
- Digital Transformation

Content managers should be able to create drafts, submit them for review, publish and archive them.

---

37. Support Module

The Support module provides customer assistance.

Users can create tickets such as:

I cannot access my course.

My order has not arrived.

I need help with my consultation.

I cannot upload my document.

Ticket workflow:

```
OPEN
IN_PROGRESS
WAITING_FOR_CUSTOMER
RESOLVED
CLOSED
```

Support agents should only access information necessary to resolve the ticket.

---

38. Notification Module

Notifications should be centralized.

The system should support:

In-app

```
You have a new application update.
```

Email

```
Your course application has been approved.
```

Future SMS/WhatsApp

These should be implemented behind a notification abstraction rather than embedded directly into each application.

---

39. Important Notification Events

Event	User
Registration	User
Email verification	User
Password reset	User
Order confirmation	Customer
Payment confirmation	Customer
Booking confirmation	Customer
Booking reminder	Customer/Consultant
Course application	Applicant/Admin
Additional information request	Applicant
Admission decision	Applicant
Enrollment	Student
Course announcement	Student
Travel status update	Applicant
Document request	Applicant
Support response	Customer

---

40. Admin Dashboard

The admin dashboard should provide a business-level overview.

Potential dashboard cards:

```
Total Users
New Users
Orders
Revenue
Pending Applications
Active Students
Upcoming Consultations
Travel Applications
Open Support Tickets
```

Graphs can later display:

- Revenue trends
- Student growth
- Application trends

- Order trends
- Consultation bookings

---

41. Administrative Pages

```
/dashboard/
/dashboard/users/
/dashboard/roles/
/dashboard/permissions/

/dashboard/store/
/dashboard/orders/
/dashboard/products/

/dashboard/lms/
/dashboard/lms/courses/
/dashboard/lms/applications/
/dashboard/lms/students/
/dashboard/lms/admissions/

/dashboard/consultations/
/dashboard/consultations/bookings/
/dashboard/consultations/consultants/

/dashboard/travel/
/dashboard/travel/applications/
/dashboard/travel/advisors/

/dashboard/projects/
/dashboard/blog/

/dashboard/payments/
/dashboard/invoices/

/dashboard/support/
/dashboard/notifications/
/dashboard/audit/
/dashboard/settings/
```

The exact URLs can be changed during implementation, but the functional separation should remain.

---

42. Permission Matrix

The following is a practical starting permission matrix for the MVP.

Legend:

- **F** = Full management
- **M** = Manage assigned records

- **V** = View
- **C** = Create/submit
- **—** = No access

Module	Super Admin	Admin	Store Manager	Instructor	Admissions	Consultant	Travel Advisor	Support	Customer	Student	Applicant
Users	F	M	—	V	V	V	V	V	Own	Own	Own
Roles	F	M	—	—	—	—	—	—	—	—	—
Website	F	M	—	—	—	—	—	—	V	V	V
Products	F	F	F	—	—	—	—	—	V	V	V
Orders	F	F	F	—	—	—	—	V	Own	—	—
Payments	F	F	V	—	V	V	V	V	Own	Own	Own
Courses	F	F	M	F	M	—	—	—	V	Enrolled	V
LMS Applications	F	F	—	V	F	—	—	—	—	—	Own
Admissions	F	F	—	—	F	—	—	—	—	Own	Own
Students	F	F	—	M	F	—	—	—	—	Own	—
Consultation	F	F	—	—	—	F	—	V	C/Own	—	—
Travel	F	F	—	—	—	—	F	V	C	—	Own
Projects	F	F	—	C	—	—	—	—	V	V	V
Blog	F	F	—	C	—	—	—	—	V	V	V
Support	F	F	—	—	—	—	—	F	Own	Own	Own
Notifications	F	M	—	M	M	M	M	M	Own	Own	Own
Audit	F	V	—	—	—	—	—	—	—	—	—
System Settings	F	M	—	—	—	—	—	—	—	—	—

This matrix is a starting point. The implementation should use **granular permissions**, not merely the broad role names shown above.

---

43. Individual User Duties

Super Administrator

The Super Administrator is responsible for the entire system.

Duties include:

- Manage administrators
- Manage roles
- Manage permissions
- Configure system settings
- Review audit logs
- Manage integrations
- Manage all business modules
- Handle security incidents
- Manage backups/infrastructure where applicable

This role should be extremely restricted.

Administrator

The Administrator manages day-to-day platform operations.

Duties include:

- Manage users
- Manage content
- Manage applications
- Manage products
- Monitor transactions
- Review reports
- Manage operational staff

The Administrator should not automatically have infrastructure-level server access.

Store Manager

Responsible for:

- Product creation
- Product updates
- Inventory
- Categories
- Orders
- Shipping status
- Product promotions

They should not have access to unrelated student or travel information.

Instructor

Responsible for:

- Course creation
- Course content
- Lessons
- Assessments
- Student progress
- Grading
- Feedback

An instructor should generally only access students enrolled in their courses.

Admissions Officer

Responsible for:

- Reviewing applications
- Reviewing documents
- Requesting information
- Scheduling interviews
- Recording decisions
- Issuing admission
- Managing enrollment workflow

Consultant

Responsible for:

- Consultation profile
- Availability
- Bookings
- Session notes
- Session completion

Travel Advisor

Responsible for:

- Applicant review
- Document review

- Application processing
- Status changes
- Applicant communication
- Case notes

Support Agent

Responsible for:

- Receiving support tickets
- Investigating issues
- Responding to customers
- Updating ticket status
- Escalating complex problems

Customer

A customer can:

- Manage profile
- Purchase products
- Make payments
- Track orders
- Book consultations
- Submit support tickets
- View their own invoices
- Receive notifications

Applicant

An applicant can:

- Create an application
- Upload documents
- Submit applications
- Respond to requests
- Track application status
- Receive admission/application notifications

Student

A student can:

- Access enrolled courses
- View lessons
- Complete assessments
- Submit assignments
- View grades
- Track progress
- Download certificates
- Receive course notifications

---

44. Common Dashboard

Rather than creating completely different systems for every user, Abraytech should use a common dashboard framework.

The content changes according to role.

For example:

Customer

```
Welcome back

Orders: 4
Bookings: 2
Open Tickets: 1
```

Student

```
My Learning

Courses: 3
Progress: 68%
Assignments: 2
Certificates: 1
```

Administrator

```
Platform Overview

Users: 2,340
Orders: 128
Revenue: £XX,XXX
Applications: 48
```

---

45. URL/Page Architecture

A developer should organize the site into three broad areas.

Public

```
/
/about/
/services/
/services/software-development/
/services/cybersecurity/
/services/artificial-intelligence/
/services/consulting/
/training/
/store/
/consultation/
/projects/
/travel-study/
/blog/
/contact/
```

Authenticated

```
/account/
/dashboard/
/dashboard/orders/
/dashboard/bookings/
/dashboard/courses/
/dashboard/applications/
/dashboard/documents/
/dashboard/notifications/
/dashboard/support/
/dashboard/profile/
```

Administration

```
/admin/
/dashboard/admin/
/dashboard/users/
/dashboard/store/
/dashboard/lms/
/dashboard/consultation/
/dashboard/travel/
/dashboard/projects/
/dashboard/content/
/dashboard/payments/
/dashboard/audit/
/dashboard/settings/
```

---

46. Database Core Entities

The initial database should contain approximately these major entities:

Domain	Core Entities
Accounts	User, Profile, Role, Permission
Website	Page, Service, FAQ, Testimonial

Domain	Core Entities
Store	Category, Product, ProductImage, Specification
Orders	Cart, CartItem, Order, OrderItem
Payments	Payment, Invoice, Transaction
LMS	Course, Module, Lesson, Application, Admission, Enrollment
Assessment	Quiz, Question, Assignment, Submission, Result
Certificate	Certificate, Verification
Consultation	Service, Consultant, Availability, Booking
Travel	Applicant, AcademicRecord, Application, Document, StatusHistory
Repository	Project, Technology, ProjectMedia
Content	BlogPost, Category, Tag
Support	Ticket, TicketMessage
Notifications	Notification, NotificationTemplate
Audit	AuditLog

---

47. Business Workflow Integrity

Every major workflow should use explicit statuses rather than Boolean fields such as:

```
is_approved = True
```

A proper status system is preferred:

```
SUBMITTED
UNDER_REVIEW
ACTION_REQUIRED
APPROVED
REJECTED
```

This creates a much more maintainable enterprise system.

---

48. Audit Trail

The following actions should be audited:

- User creation
- Role changes
- Permission changes
- Application approval
- Application rejection
- Admission issuance
- Payment updates
- Refunds
- Order status changes

- Document access where appropriate
- Travel application status changes
- Administrative settings changes

An audit entry should record:

```
Who
What
When
Where
Which object
Previous state
New state
Additional metadata
```

---

49. Security Requirements

The developer should implement at minimum:

- HTTPS
- CSRF protection
- Secure cookies
- Password hashing
- Email verification
- Permission enforcement
- Object-level authorization
- Rate limiting
- Input validation
- File validation
- Secure document storage
- Webhook verification
- Secret management
- Security headers
- Audit logging
- Error monitoring
- Regular backups

Sensitive documents must never be exposed through predictable public media URLs.

---

50. JavaScript Responsibilities

JavaScript should be used primarily for progressive enhancement and interactive experiences.

Examples:

- Mobile navigation
- Dropdowns

- Modal dialogs
- Form validation feedback
- Dynamic filters
- Shopping cart updates
- Availability calendar
- Notifications
- File-upload progress
- Dashboard charts
- Search suggestions
- AJAX/Fetch operations

The core business rules must remain on the Django backend.

---

51. Tailwind CSS Responsibilities

Tailwind should provide a consistent design system.

Reusable components should include:

```
Button
Card
Modal
Alert
Badge
Table
Form
Input
Select
Dropdown
Tabs
Pagination
Breadcrumb
Navbar
Sidebar
DashboardCard
EmptyState
LoadingState
```

The developer should avoid building every page with unrelated CSS.

---

52. Responsive Design

Every important workflow should work on:

```
Mobile
Tablet
Laptop
Desktop
```

```
Large Desktop
```

Especially:

- Registration
- Login
- Checkout
- Course application
- Document upload
- Consultation booking
- Student dashboard
- Travel application
- Admin dashboard

---

53. SEO

The public website should have:

- Semantic HTML
- Proper title tags
- Meta descriptions
- Open Graph metadata
- Structured URLs
- Sitemap
- Robots configuration
- Canonical URLs
- Schema markup where appropriate
- Optimized images
- Fast page rendering

Important SEO pages include:

```
Software Development
Cybersecurity
AI Services
Technology Training
Technology Consulting
Study Abroad Services
Technology Store
```

---

54. Analytics

The platform should track meaningful business events rather than merely page views.

Examples:

```
course_viewed
```

```
course_application_started
course_application_submitted
product_viewed
product_added_to_cart
checkout_started
purchase_completed
consultation_started
consultation_booked
contact_form_submitted
project_viewed
```

Analytics must respect applicable privacy requirements.

---

55. Email and Notification Architecture

Applications should not individually implement email sending.

Instead:

```
Business Event
      ↓
Notification Service
      ↓
Notification Record
      ↓
Celery
      ↓
Email Provider
```

This allows the system to handle email asynchronously.

For example, generating an admission letter should not make the user wait for an SMTP server before the HTTP request completes.

---

56. Celery Background Jobs

The following should be suitable candidates for background tasks:

Task	Background?
Email sending	Yes
Booking reminders	Yes
Admission letter generation	Yes
Certificate generation	Yes
Invoice generation	Yes
Report generation	Yes
Notification processing	Yes

Task	Background?
Image processing	Yes
Cleanup tasks	Yes
Payment webhook processing	Often
Simple CRUD	No

---

57. API Layer

The API should be designed from the beginning even if the MVP primarily uses Django templates.

Recommended:

```
/api/v1/
```

The API will make future development easier for:

- Mobile applications
- External integrations
- React frontend
- AI services
- Partner systems

The web application should not be tightly coupled to future API consumers.

---

58. Mobile App Readiness

Although the first MVP is a web application, the architecture should make a future Abraytech mobile application possible.

Potential mobile functionality:

- Student learning
- Consultation booking
- Order tracking
- Travel application tracking
- Notifications
- Messaging

The same Django backend/API can eventually serve the mobile application.

---

59. Recommended Development Sequence

A developer should build Abraytech in this order:

Phase 1 — Foundation

```
Django
PostgreSQL
Custom User
Authentication
RBAC
Base Template
Tailwind
Core utilities
```

Phase 2 — Public Website

```
Homepage
About
Services
Projects
Training
Store
Consultation
Travel
Blog
Contact
Legal
```

Phase 3 — Customer Platform

```
Dashboard
Profile
Notifications
Support
Payments
```

Phase 4 — Store

```
Products
Categories
Cart
Checkout
Stripe
Orders
Inventory
```

Phase 5 — LMS

```
Courses
Applications
Documents
Admissions
Enrollment
Lessons
```

```
Assessments
Certificates
```

Phase 6 — Consultation

```
Services
Consultants
Availability
Booking
Payment
Reminders
```

Phase 7 — Travel

```
Applicant
Application
Documents
Advisor
Status tracking
Notifications
```

Phase 8 — Enterprise Features

```
Reporting
Audit
Advanced analytics
Advanced notifications
AI
Calendar
Video conferencing
```

---

60. MVP vs Future Features

Not everything should be built in version 1.

Feature	MVP	Future
Corporate website	✓	
Authentication	✓	
RBAC	✓	
Store	✓	
Stripe	✓	
Orders	✓	
LMS applications	✓	
Admissions	✓	
Online courses	✓	
Consultation booking	✓	
Project repository	✓	
Travel/study applications	✓	
Notifications	✓	
Support tickets	✓	

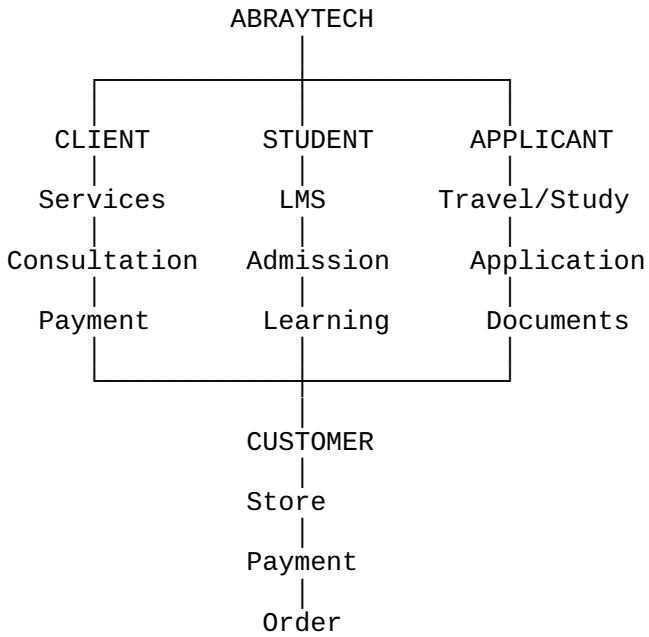
Feature	MVP	Future
Blog	✓	
Advanced analytics		✓
AI Tutor		✓
AI cybersecurity assistant		✓
Mobile app		✓
WhatsApp integration		✓
Advanced calendar sync		✓
Video conferencing integration		✓
Multi-tenant institutions		✓
Advanced CRM		✓
Advanced accounting		✓

---

61. Recommended MVP Definition

The first production release should prove that Abraytech can operate as a **real technology business**, rather than attempting to implement every possible future feature.

The MVP should therefore successfully support this complete journey:



The public website brings users into this ecosystem, while the authenticated platform provides the actual business functionality.

---

62. The Developer's Core Implementation Rule

The most important technical principle for this project should be:

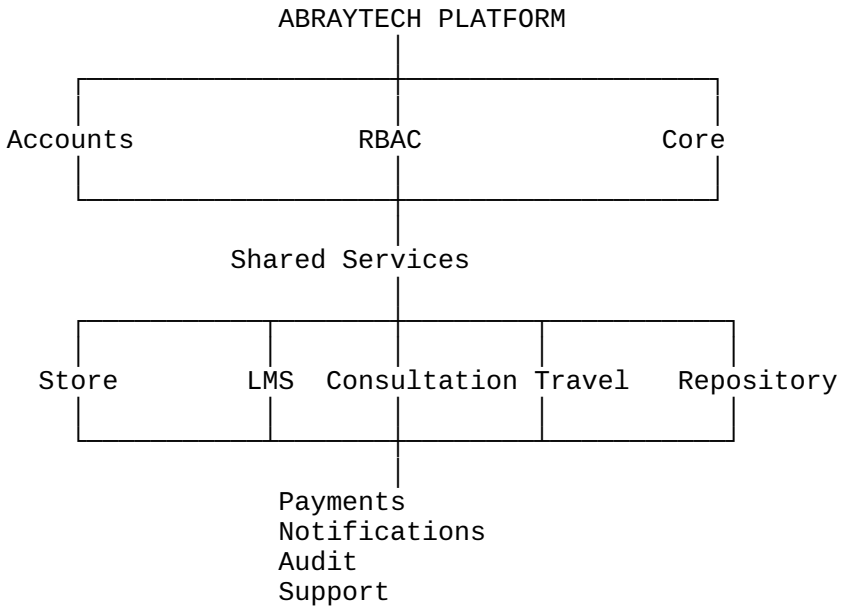
Build the shared platform once, then build business modules on top of it.

Do **not** create separate authentication systems for the Store, LMS, Consultation and Travel applications.

Do **not** create separate dashboards for every module unless the UX genuinely requires it.

Do **not** duplicate users, payments, notifications or permissions across applications.

Instead:



This is the structure that will allow Abraytech to grow from an MVP into a much larger technology platform.

---

63. Final Developer Deliverable

At the end of MVP development, the developer should deliver a production-ready system containing:

Area	Required Outcome
Frontend	Responsive Tailwind-based multi-page website
Backend	Modular Django application

Area	Required Outcome
Database	PostgreSQL
Authentication	Secure custom user system
Authorization	RBAC + granular permissions
Store	Full basic e-commerce
Payments	Verified Stripe integration
LMS	Application → admission → enrollment → learning
Consultation	Availability → booking → payment
Travel	Application → documents → processing
Repository	Public project portfolio
Blog	Content publishing
Support	Ticket management
Notifications	In-app + transactional email
Audit	Administrative activity tracking
API	Versioned `/api/v1/` foundation
Background tasks	Celery + Redis
UI	Tailwind + JavaScript
Security	Production security hardening
Testing	Unit, integration and workflow tests
Deployment	Nginx + Gunicorn + PostgreSQL + Redis
Documentation	Installation, configuration and developer documentation

64. The Complete Abraytech Product in One Sentence

Abraytech should be built as a unified Django-powered technology business platform where a visitor can discover Abraytech's services, buy technology products, apply for technology training, become a student, book professional consultations, showcase or consume technology projects, and receive international study application support—all through one secure account, one consistent interface, one permission system and one extensible business platform.

This consolidated specification can serve as the **developer's master build reference** for the MVP, while the detailed PRD/SRS/Architecture documents should remain the formal source of truth for individual technical decisions.